



Building Java Programs
A BACK TO BASICS APPROACH 5e

Computer Science

Implementation - Hashtable

Instructor: Varik Hoang

Java

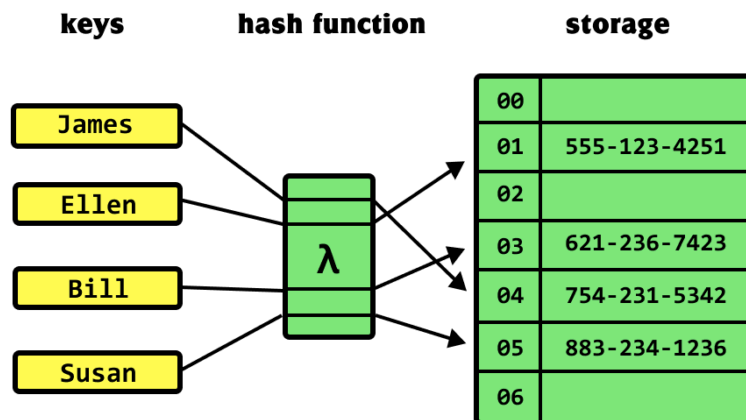


Stuart Reges / Marty Stepp



Objectives

- The basic idea of hashing
- The hash table and hash function
- The hash code as an index to the hash table
- The collisions and open addressing
- Type of hashing
 - Open Addressing
 - Linear Probing
 - Quadratic Probing
 - Double Hashing
 - Closed Addressing
 - The Separate Chaining





What Is Hashing?

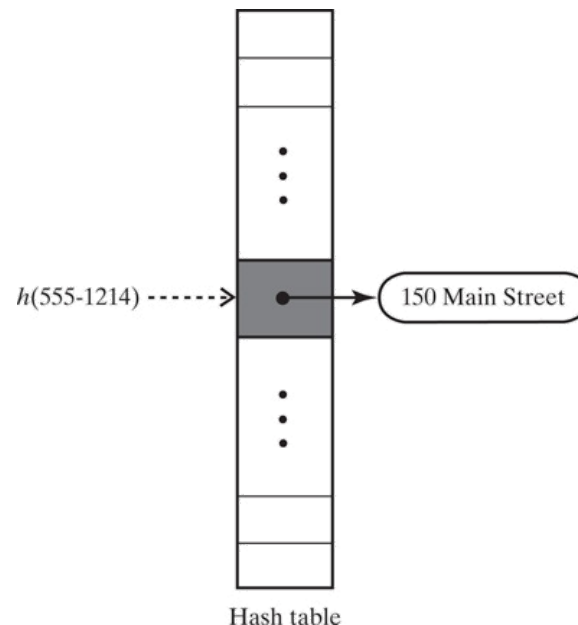
- Access any entry in an array directly if we know its index
- **Hashing** is a technique to determine index using key/map without searching
- A **hash function** takes a key and produces the integer index in the hash table

$h(555-1264)$

- Referencing the appropriate street address is a case for a **perfect hash function**

```
Algorithm add(key, value)
    index = h(key)
    hashTable[index] = value
```

```
Algorithm getValue(key)
    index = h(key)
    return hashTable[index]
```





What Is Hashing? (cont.)

- The perfect hash function needs a hash table with **10,000** different indices
 - The hash table *is always full* if every phone number in **555** exchange is assigned
- Most hash tables *are not full* and can even be **sparse**
 - Need a different hash function if we wanted to use a *smaller hash table*

```
Algorithm getHashIndex(phoneNumber)
    // Returns an index to an array of tableSize elements.
    i = last four digits of phoneNumber
    return i % tableSize
```

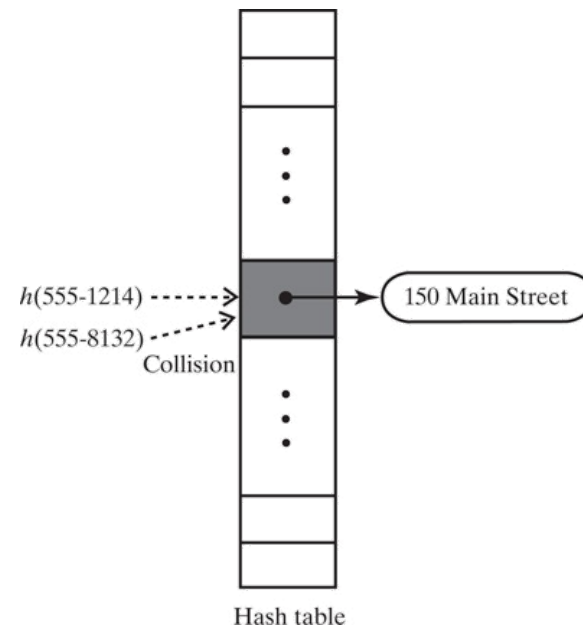
- The hash function performs two steps:
 - 1 Convert the key to an integer called the **hash code**
 - 2 **Compress** the hash code into the range of indices for the hash table



What Is Hashing? (cont.)

- The algorithm **getHashIndex** is not a perfect hash function
- Some phone numbers will map into the **same index**
 - **getHashIndex("555-1264")** map into **52**
 - **getHashIndex("555-8132")** map into **52**
- We call this occurrence a **collision**

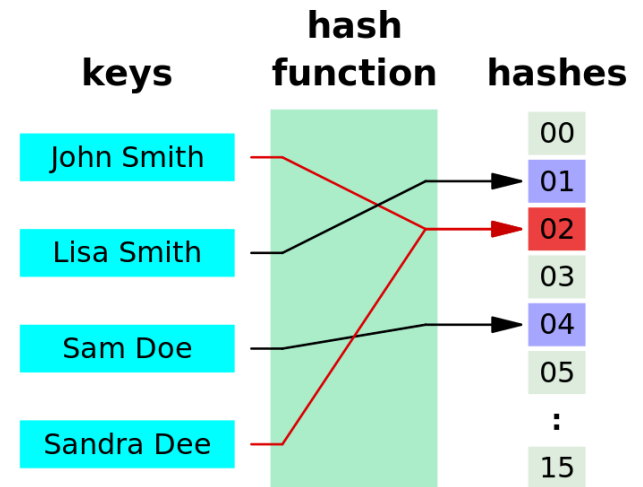
```
Algorithm getHashIndex(phoneNumber)
    // Returns an index to an array of tableSize elements.
    i = last four digits of phoneNumber
    return i % tableSize
```





Hash Functions

- Any good hash function should:
 - **Minimize** collisions
 - Be **fast** to compute
- Hash function compresses hash code into an integer that is suitable as an index
 - To reduce the chance of a collision by **distributing entries** *uniformly* in table
 - To have a key can be either a primitive type or an instance of a class





Computing Hash Codes

- **Object** has a method **hashCode** that returns an integer hash code
- The **hashCode** returns an **int** value based on memory address of the object
- Guidelines for the method **hashCode**:
 - A class should override **hashCode** if a class overrides the method **equals**
 - The **hashCode** must return **same** value for both objects if the method **equals** considers two objects equal
 - The **hashCode** must return **same** hash code if we call **hashCode** more than once during the execution of a program
 - The **hashCode** must return **different** hash code during another execution of the same program
- A perfect hash function would avoid duplicate hash codes lead to **collisions**



Compressing a Hash Code

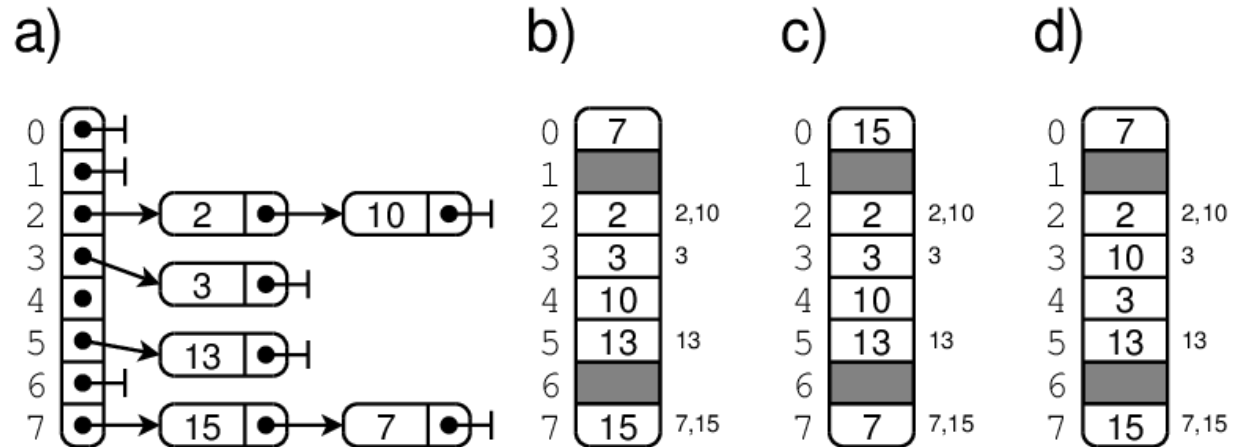
- The most common way to use mod (%) operator
 - A positive hash code **c**
 - A positive integer **n**
 - **c % n** is ideal for the index of a hash table
- **c % n** has the same **parity** as **c**
 - **c % n** is even if **c** is even
 - **c % n** is odd if **c** is odd
- **c % n** provides values that are distributed throughout the index range **0** through **n-1** when **n** is a **prime number**

```
private int getHashIndex(Key key)
{
    int hashIndex = key.hashCode() % hashTable.length;
    if (hashIndex < 0)
        hashIndex = hashIndex + hashTable.length;
    return hashIndex;
}
```



Resolving Collisions

- Open Addressing
 - Linear Probing
 - Quadratic Probing
 - Double Hashing
- Separate Chaining

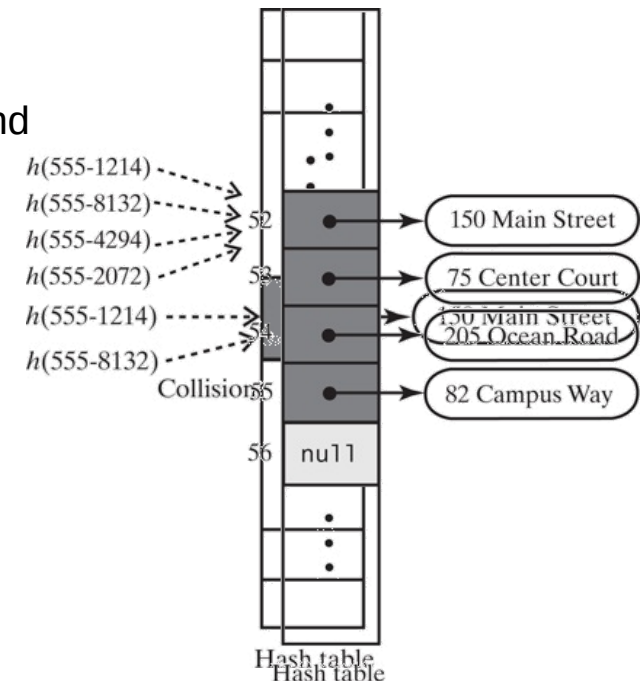




Open Addressing with Linear Probing (OALP)

- Locating an open element in the hash table is called **probing**
- With **linear probing**, if a collision occurs at **table[k]**
 - Check **table[k + 1]** (**i = 1**) if available, if not
 - Check **table[k + 2]** (**i = 2**) if available, and so on
- We consider in search make up the **probe sequence**
 - Search continues at beginning of the table if reaches the end
 - First element comes immediately after last element
- **Linear probing** resolves a collision during hashing
 - Check consecutive elements for next open one

```
addressBook.add("555-1264", "150 Main Street");
addressBook.add("555-8132", "75 Center Court");
addressBook.add("555-4294", "205 Ocean Road");
addressBook.add("555-2072", "82 Campus Way");
```

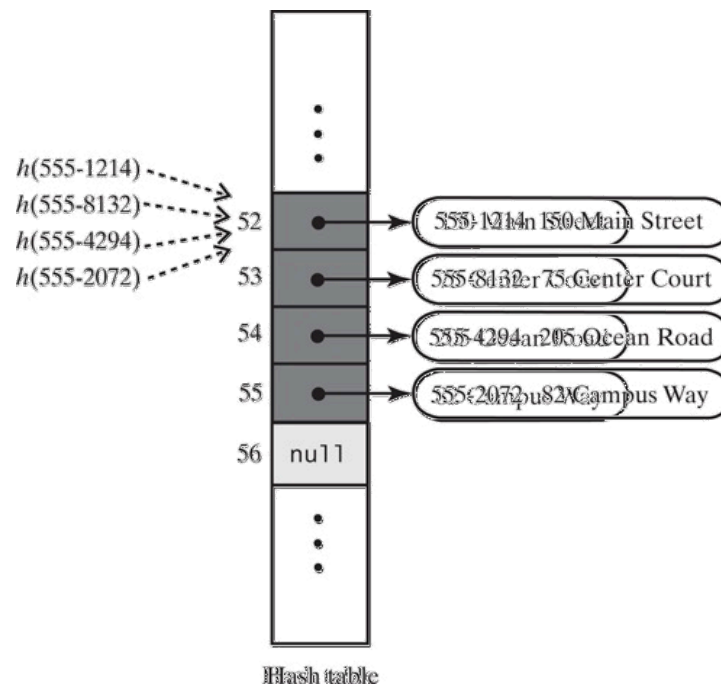




OALP - Retrieval

- How do we retrieve the street address associated with the last key we added?
 - **getHashIndex("555-2072")** is **52**
 - **getValue** searches consecutive elements beginning at **table[52]**
- How can we tell which street address is the right one?
 - We can't, unless we *package a key* with its value
 - Dictionary implementations provided a class **Entry**
- Follow the same probe sequence that added

```
String streetAddress = addressBook.getValue("555-2072");
```

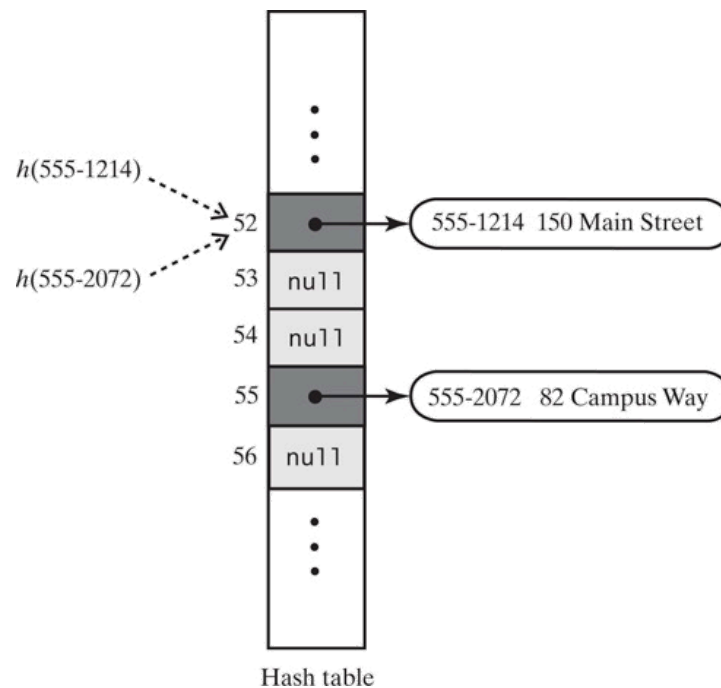




OALP - Removal

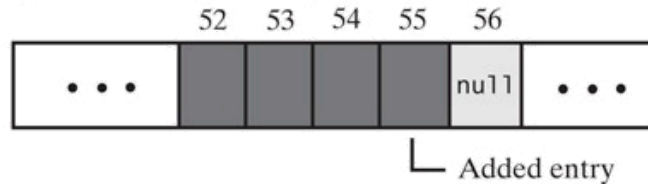
- The simplest way to remove an entry from a table is to replace it with **null**, ...
 - But break the searching entirely
- We need to distinguish among three kinds of elements in the hash table
 - Occupied - the element references an entry in dictionary
 - Empty - the element contains **null**
 - Available - the element was removed from dictionary
- The method **remove**
 - Should not replace an entry with **null**
 - But mark location as available (**tombstone**)

```
addressBook.remove("555-8132");  
addressBook.remove("555-4294");
```

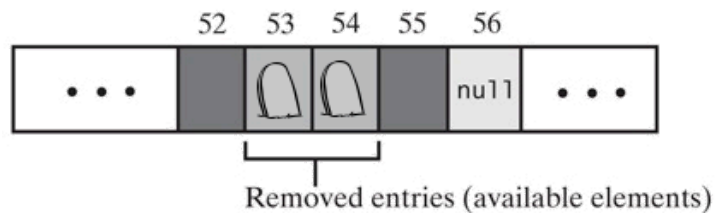


OALP - Removal (cont.)

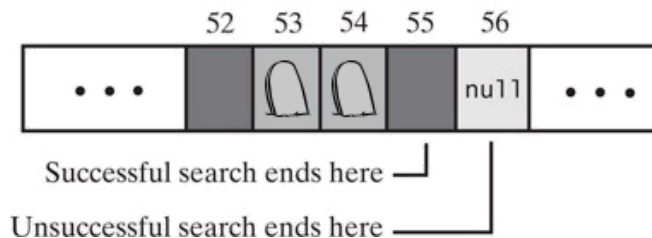
(a) After adding an entry



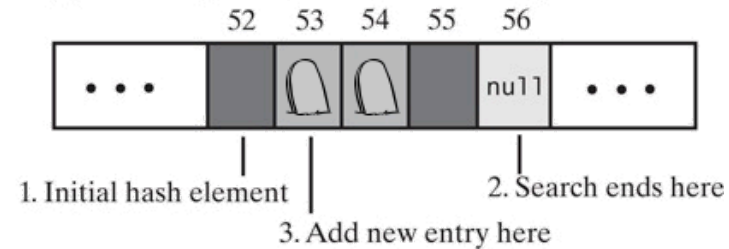
(b) After removing two entries



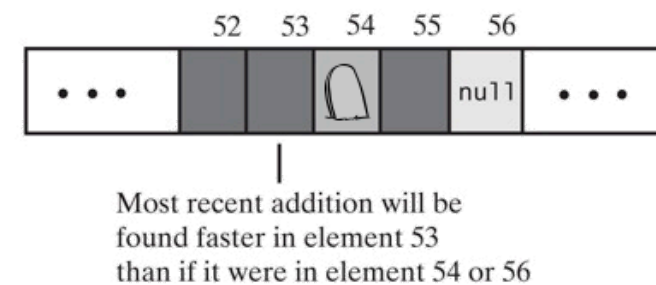
(c) After a search



(d) Searching for a place to add an entry



(e) After an addition to a formerly occupied element



Dark gray = occupied with current entry
Medium gray = available element (tombstone)
Light gray = empty element (contains null)





OALP - The **probe** Method & Clustering

- The private method **probe**(key, index) looks for key along a probe sequence
 - The search ignores elements that are in the available (**tombstone**) state
 - Continues until it locates either key or **null**
 - Returns the index of a table element
 - References an entry containing key
 - Is available for an addition to the table
- Collisions are resolved with linear probing cause groups of consecutive elements
 - Each group is called a **cluster**
 - the phenomenon is known as **primary clustering**
- Each cluster is a **probe sequence** that we must search when adding, removing, or retrieving a table entry



OALP - The **probe** Method - Pseudocode

```
Algorithm probe(key, index)
    // Searches the probe sequence from current index
    // Returns the index of either the element
    // containing key or an available element in the hash table.
    while (key is not found and hashTable[index] is not null)
    {
        if (hashTable[index] references an entry in the dictionary)
        {
            if (the entry in hashTable[index] contains key)
                Exit loop
            else
                index = next probe index
        }
        else // hashTable[index] is available
        {
            if (this is the first available element encountered)
                availableStateIndex = index
            index = next probe index
        }
    }
    if (key is found or an available element was not encountered)
        return index
    else
        return availableStateIndex // Index of first entry removed
```



OALP - Addition - Example (1)

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

22

Insert **38, 19, 12, 8, 109**, and **10** to the hash table

- With capacity **10**
- Using **linear probing (+1)**



OALP - Addition - Example (2a)

0	
1	
2	
3	
4	
5	
6	
7	
8	38
9	

Insert **38**

— **getHashIndex(38) = 38 % 10 = 8**



OALP - Addition - Example (2b)

0	
1	
2	
3	
4	
5	
6	
7	
8	38
9	

24

Insert **38**

— We place **38** in because **table[8]** was empty



OALP - Addition - Example (3a)

0	
1	
2	
3	
4	
5	
6	
7	
8	38
9	19

Insert **19**

— **getHashIndex(19) = 19 % 10 = 9**



OALP - Addition - Example (3b)

0	
1	
2	
3	
4	
5	
6	
7	
8	38
9	19

Insert **19**

— We place **19** in because **table[9]** was empty



OALP - Addition - Example (4a)

0	
1	
2	12
3	
4	
5	
6	
7	
8	38
9	19

Insert **12**

— **getHashIndex(12) = 12 % 10 = 2**



OALP - Addition - Example (4b)

0	
1	
2	12
3	
4	
5	
6	
7	
8	38
9	19

28

Insert **12**

— We place **12** in because **table[2]** was empty

OALP - Addition - Example (5a)

0	
1	
2	12
3	
4	
5	
6	
7	
8	38
9	19

Insert 8

- $\text{getHashIndex}(8) = 8 \% 10 = 8$
- We **cannot** place 8 in because **table[8]** is **not** empty
- We move 8 to next index for $\text{probe}(8, 8) = 9$



OALP - Addition - Example (5b)

0	
1	
2	12
3	
4	
5	
6	
7	
8	38
9	19
	8

Insert **8**

- We **cannot** place **8** in because **table[9]** is **not** empty
- We move **8** to next **probe sequence**



OALP - Addition - Example (5c)

0		8
1		
2	12	
3		
4		
5		
6		
7		
8	38	
9	19	

Insert **8**

— We move **8** to next index **0** for **probe(8, 9) = 0**



OALP - Addition - Example (5d)

0	8
1	
2	12
3	
4	
5	
6	
7	
8	38
9	19

32

Insert **8**

— We place **8** in because **table[0]** was empty



Questions & Answer





Thank You!

A black and silver fountain pen with a silver nib, lying horizontally below the 'Thank You!' text. The pen has a black barrel with silver-colored rings and a silver-colored clip.